



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

INGENIERÍA EN COMPUTACIÓN

Laboratorio de Organización de Arquitectura de Computadoras

Reporte - Construcción de máquinas de Estados usando memorias: Direccionamiento Implícito

NOMBRE DE LOS INTEGRANTES:
Omar Alfonso Zendejas Labias Grupo de Teoría: 04
Emiliano García Olvera Grupo de Teoría: 02

NOMBRE DEL PROFESOR: ING. KAREN PÉREZ

GRUPO: 06 EQUIPO: 9

SEMESTRE: 2027-1.

FECHA DE ENTREGA: 5 de octubre de 2026

Objetivo

Construir una máquina de estados finita a partir de una carta ASM utilizando una memoria como elemento central del diseño, empleando el método de direccionamiento implícito. A diferencia del direccionamiento por trayectoria y del direccionamiento entrada-estado, aquí la dirección de memoria se forma únicamente con el estado presente. El estado siguiente se determina con un contador que avanza en uno o carga el campo de liga, según el valor de la variable sensada.

Para ello se diseñó una carta ASM de siete estados, tres entradas (X, Y y Z) y cuatro salidas (dos con lógica estándar y dos con lógica negada), con salidas condicionales en cinco decisiones. Se determinó el tamaño de la memoria, se obtuvo su contenido completo, se implementó el sistema en VHDL sobre la tarjeta DE10-Lite mediante Quartus Prime, y se verificó su funcionamiento por simulación y en hardware.

Introducción

En el direccionamiento implícito, la memoria no almacena dos ligas como en el método entrada-estado, sino una sola **liga** de salto. Un campo de **prueba** selecciona la variable de entrada que se sensa en el estado, y el campo **VF** (verdadero-falso) indica el valor que debe tener esa variable para realizar el salto [1].

Una compuerta XNOR compara la variable sensada (Q) con VF. Si ambos valores coinciden, se genera la señal de **carga**, y el contador toma el valor del campo de liga. Si no coinciden, se genera la señal de **incremento** (una compuerta XOR entre Q y VF), y el contador avanza al estado siguiente en secuencia. La relación se resume en la siguiente tabla.

VF	Q	Incrementa	Carga
0	0	0	1
0	1	1	0
1	0	1	0
1	1	0	1

Como consecuencia, si una decisión no cumple la condición de salto, el estado siguiente debe ser el código del estado presente más uno. Esto obliga a que las ramas de incremento apunten a códigos consecutivos.

Las salidas condicionales se manejan desdoblando el campo de salidas en dos conjuntos: las

salidas F, activas cuando la variable sensada vale 0 (incremento), y las **salidas V**, activas cuando vale 1 (carga). Un multiplexor de 2 a 1 selecciona entre ambos conjuntos con la señal de carga.

El tamaño de la memoria se calcula con

$$\text{Memoria} = 2^n \times m \tag{1}$$

donde n es el número de bits de estado y m el número de bits por localidad.

Desarrollo

Planteamiento: carta ASM

La carta ASM se diseñó primero a mano, como se muestra en la figura 2. Después se trasladó a la forma digital de la figura 1. Tiene siete estados (EST0 a EST6), tres entradas (X, Y y Z) y cuatro salidas: S_1 y S_2 con lógica estándar, y $\overline{U_1}$ y $\overline{U_2}$ con lógica negada. En la carta, $\overline{U_1}$ y $\overline{U_2}$ aparecen como U1 y U2 en el diseño manual.

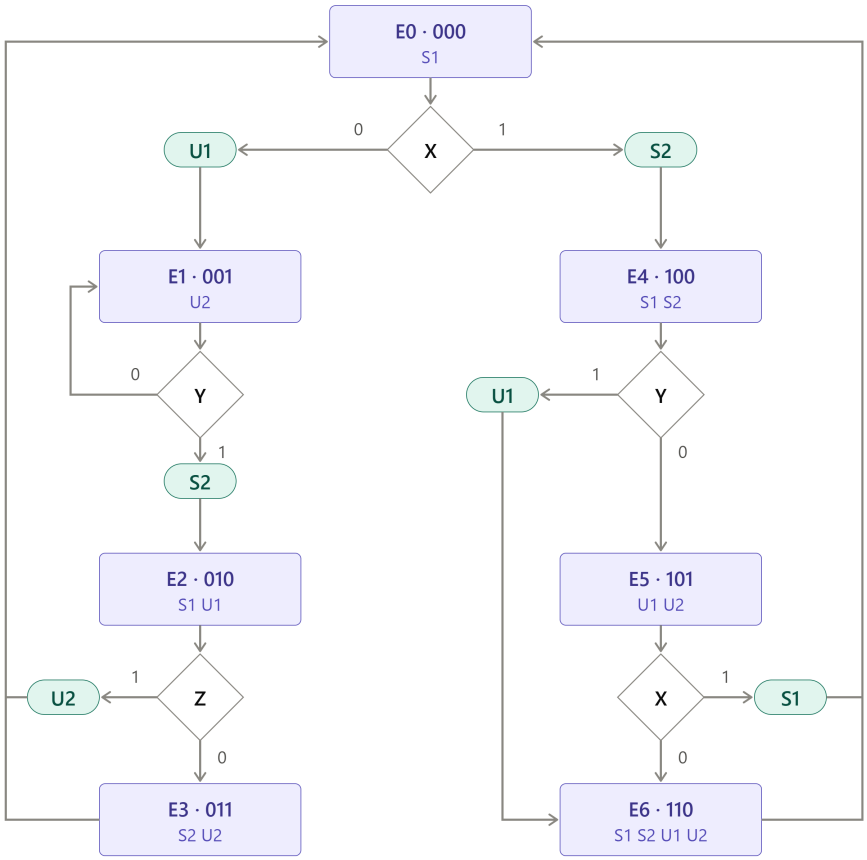


Figura 1: Carta ASM propuesta: siete estados, tres entradas y cuatro salidas, con salidas condicionales en las decisiones de EST0, EST1, EST2, EST4 y EST5.

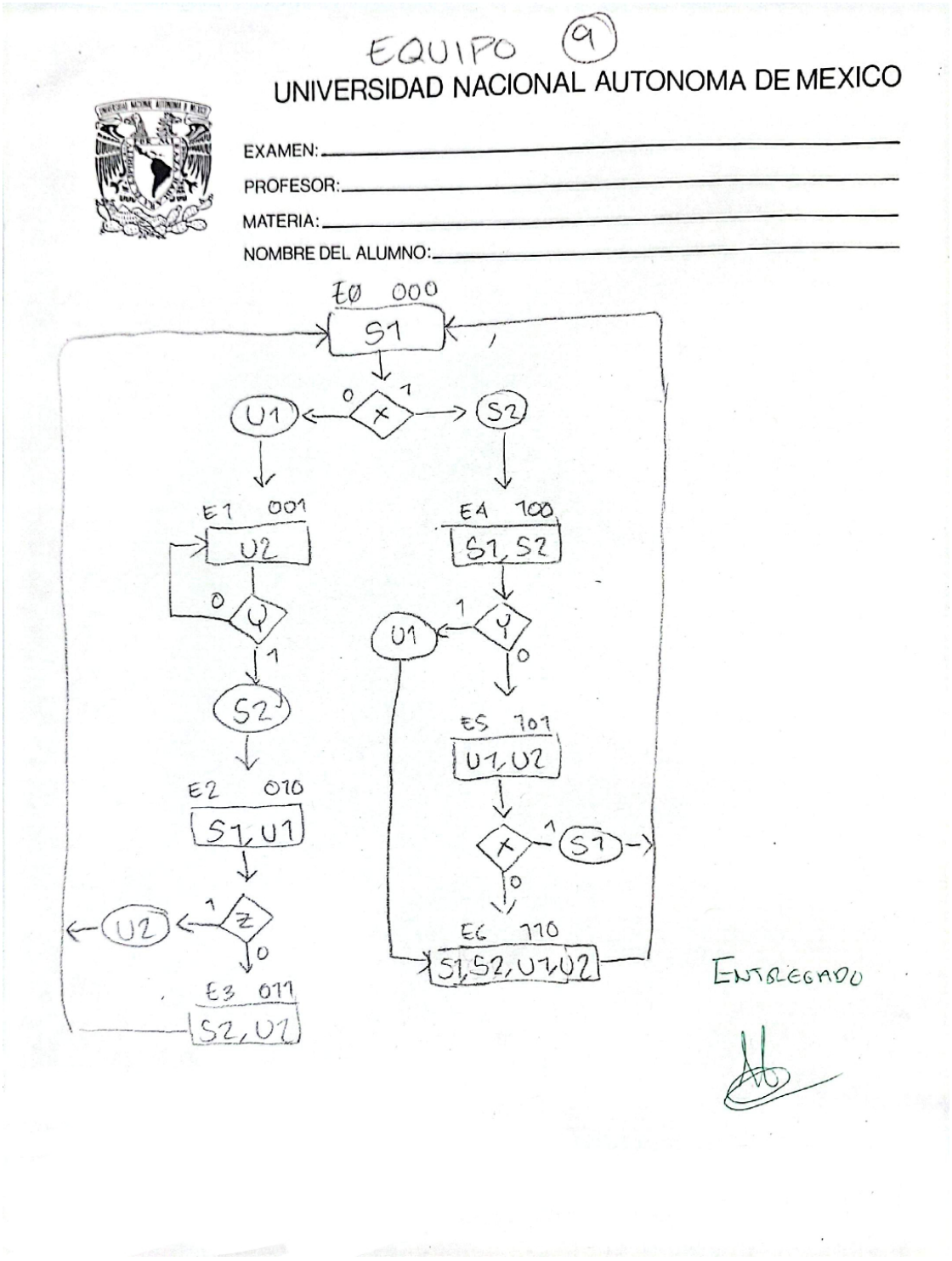


Figura 2: Evidencia del diseño manual de la carta ASM, con la asignación de códigos de estado.

Con siete estados se requieren $\lceil \log_2 7 \rceil = 3$ bits de estado, por lo que hay ocho localidades y una dirección sin utilizar (111). La variable sensada se codifica en el campo de prueba. Las transiciones incondicionales de EST3 y EST6 hacia EST0 se resuelven con una prueba sobre la entrada `aux`,

conectada a GND. Con VF=0, esa prueba siempre provoca carga. La tabla 1 resume la codificación.

Tabla 1: Codificación de la variable sensada en el campo de prueba.

Prueba	**Variable**
`000`	X
`001`	Y
`010`	Z
`011`	W (no usada por la carta)
`100`	`aux` (GND, siempre 0: salto incondicional)

La tabla 2 presenta la asignación de códigos, las salidas incondicionales de cada estado y la condición de salto. Las salidas condicionales se describen a continuación.

Tabla 2: Asignación binaria de los estados, con sus salidas incondicionales y su condición de salto.

Estado	**Código**	**Salidas incondicionales**	**Prueba**	**VF**	**Liga**
EST0	`000`	S_1	X	1	EST4 (`100`)
EST1	`001`	$\overline{U_2}$	Y	0	EST1 (`001`)
EST2	`010`	S_1, $\overline{U_1}$	Z	1	EST0 (`000`)
EST3	`011`	S_2, $\overline{U_2}$	aux	0	EST0 (`000`)
EST4	`100`	S_1, S_2	Y	1	EST6 (`110`)
EST5	`101`	$\overline{U_1}$, $\overline{U_2}$	X	1	EST0 (`000`)
EST6	`110`	S_1, S_2, $\overline{U_1}$, $\overline{U_2}$	aux	0	EST0 (`000`)

Las salidas condicionales son las siguientes:

- EST0: $\overline{U_1}$ se activa cuando X=0 (incremento a EST1), y S_2 cuando X=1 (carga a EST4).
- EST1: S_2 se activa cuando Y=1 (incremento a EST2). Cuando Y=0, el estado se mantiene en sí mismo.
- EST2: $\overline{U_2}$ se activa cuando Z=1 (carga a EST0). Cuando Z=0, incrementa a EST3.
- EST4: $\overline{U_1}$ se activa cuando Y=1 (carga a EST6). Cuando Y=0, incrementa a EST5.
- EST5: S_1 se activa cuando X=1 (carga a EST0). Cuando X=0, incrementa a EST6.

Para que cada decisión use el mecanismo de incremento o de carga, las ramas de incremento deben apuntar a códigos consecutivos: EST0→EST1, EST1→EST2, EST2→EST3, EST4→EST5

y EST5→EST6. Las transiciones incondicionales de EST3 y EST6 hacia EST0 se resuelven con la prueba sobre `aux`.

Respecto a las salidas negadas, se conserva el criterio de las prácticas anteriores: en la memoria se almacena el nivel físico de la señal, de modo que $\overline{U_1}$ y $\overline{U_2}$ valen `0` cuando están activas y `1` cuando están en reposo.

Tamaño de la memoria necesaria

La dirección se forma únicamente con los 3 bits del estado presente, por lo que $n = 3$ y la memoria tiene $2^3 = 8$ localidades. Cada localidad contiene los campos del método:

Campo	**Bits**
Parte de prueba (X, Y, Z y `aux`)	3
VF	1
Liga	3
Salidas F (incremento)	4
Salidas V (carga)	4
Total m	**15**

Por lo tanto

$$\text{Memoria} = 2^3 \times 15 = 8 \times 15 = 120 \text{ bits},$$

es decir, 8 localidades de 15 bits cada una. Siete de ellas corresponden a los estados de la carta ASM. La localidad `111` no corresponde a ningún estado; se le asignó un contenido de seguridad con prueba `aux`, VF=0 y liga a EST0, de modo que si la máquina alcanzara ese código retornaría a EST0 en el siguiente flanco de reloj.

En el cálculo manual previo se consideró una prueba de 2 bits (sólo X, Y y Z), lo que daba 14 bits por localidad y 112 bits en total. Esa estimación no contemplaba el salto incondicional, que requiere la entrada `aux`. Por esa razón el diseño final usa prueba de 3 bits.

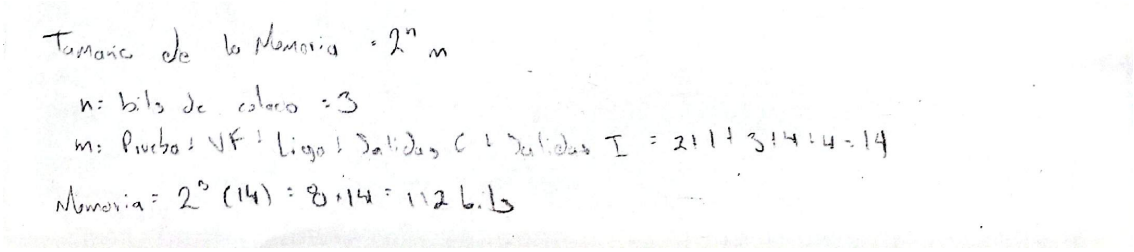


Figura 3: Cálculo manual inicial del tamaño de la memoria, con prueba de 2 bits.

Contenido de la memoria

La palabra se organiza como

$$\text{dato} = \underbrace{\text{Prueba}}_{(14:12)} \parallel \underbrace{\text{VF}}_{(11)} \parallel \underbrace{\text{Liga}}_{(10:8)} \parallel \underbrace{\text{Salidas F}}_{(7:4)} \parallel \underbrace{\text{Salidas V}}_{(3:0)}$$

Dentro de cada campo de salidas, los bits se ordenan como $S_1 S_2 \overline{U_1 U_2}$. La tabla 3 presenta las ocho localidades, tal como se escribieron en el código `memory.vhd`.

Tabla 3: Contenido de la memoria con direccionamiento implícito. La dirección 111 no corresponde a un estado de la carta: su contenido de seguridad regresa a EST0.

Dirección	Prueba	VF	Liga	Salidas F	Salidas V
000 (EST0)	000	1	100	1001	1111
001 (EST1)	001	0	001	0010	0110
010 (EST2)	010	1	000	1001	1000
011 (EST3)	100	0	000	0110	0110
100 (EST4)	001	1	110	1111	1101
101 (EST5)	000	1	000	0000	1000
110 (EST6)	100	0	000	1100	1100
111 (sin uso)	100	0	000	0011	0011

Implementación en VHDL

El sistema se describió en VHDL, repartido en nueve módulos más la entidad de nivel superior `practica5`. La figura 4 presenta la misma interconexión vista como esquemático de bloques en Quartus. El contador entrega la dirección `SAL[2..0]`, que direcciona la memoria. La palabra leída se separa en prueba, VF, liga y salidas F y V. El selector de entradas `mux5` toma la prueba y entrega la variable sensada; la compuerta XNOR la compara con VF y genera la señal de carga, que gobierna el contador y el multiplexor de liga. El multiplexor de salidas, en cambio, conmuta con el valor

crudo de la variable sensada, de modo que las salidas falsas se aplican cuando esa variable vale 0 y las verdaderas cuando vale 1.

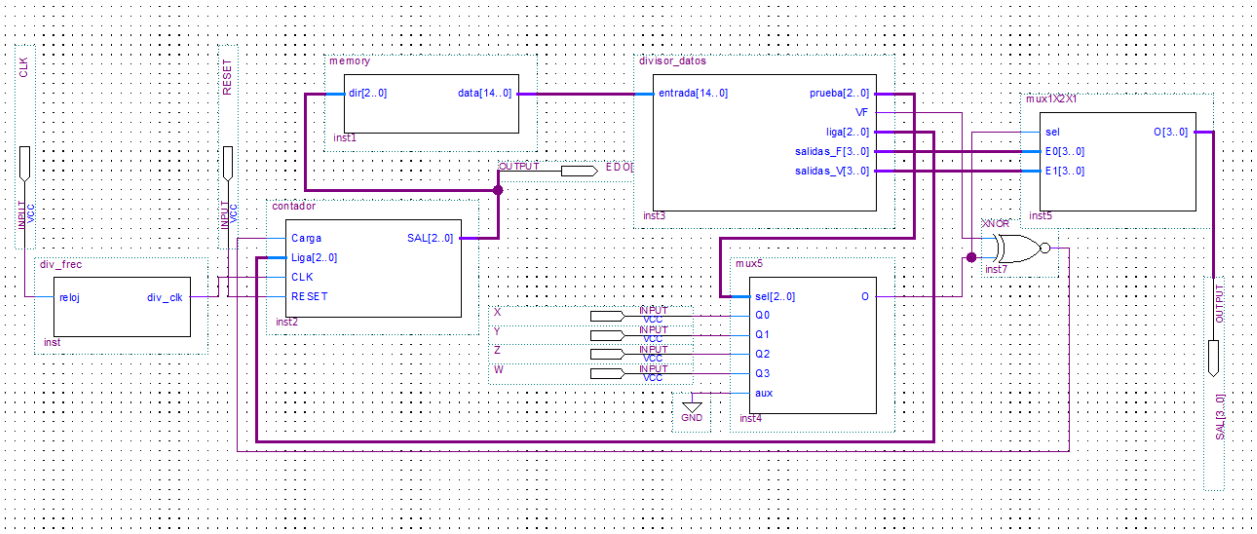


Figura 4: Esquemático en Quartus con la interconexión de los módulos.

El módulo `memory` implementa la ROM de 8 palabras de 15 bits.

```vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity memory is
  Port( dir : in STD_LOGIC_VECTOR(2 downto 0);
      data : out STD_LOGIC_VECTOR(14 downto 0)
      );
end memory;

architecture Behavioral of memory is
  type mem is array(0 to 7) of std_logic_vector(14 downto 0);
  signal internal_mem : mem;

begin
  --Llenado de memoria. Direccionamiento implicito:
  --  Palabra = Prueba(3) & VF(1) & Liga(3) & SalFalsas(4) & SalVerdaderas(4)
  --  Memoria = 2^3 x 15 = 120 bits
  --
  --  Prueba:  000=X  001=Y  010=Z  011=W  100=aux(GND, constante 0)
  --  VF:      valor de la variable sensada con el que se CARGA la liga;
  --           Carga = XNOR(variable, VF). Si Carga=0 el contador incrementa.
```

```
  --  Salidas: S1 S2 U1 U2
  --           S1 y S2: logica estandar (1 = activa, 0 = inactiva)
  --           U1 y U2: logica negada   (0 = activa, 1 = inactiva)
  --           SalFalsas se aplican cuando la variable sensada vale 0,
  --           SalVerdaderas cuando vale 1.

  --                        Prueba  VF    Liga    SalF    SalV
  internal_mem(0) <= "000" & "1" & "100" & "1001" & "1111"; --E0: S1;
     X=0 +1->E1 (U1) | X=1 carga E4 (S2)
  internal_mem(1) <= "001" & "0" & "001" & "0010" & "0110"; --E1: U2;
     Y=0 carga E1    | Y=1 +1->E2 (S2)
  internal_mem(2) <= "010" & "1" & "000" & "1001" & "1000"; --E2: S1 U1;
     Z=0 +1->E3      | Z=1 carga E0 (U2)
  internal_mem(3) <= "100" & "0" & "000" & "0110" & "0110"; --E3: S2 U2;
     incondicional carga E0
  internal_mem(4) <= "001" & "1" & "110" & "1111" & "1101"; --E4: S1 S2;
     Y=0 +1->E5      | Y=1 carga E6 (U1)
  internal_mem(5) <= "000" & "1" & "000" & "0000" & "1000"; --E5: U1 U2;
     X=0 +1->E6      | X=1 carga E0 (S1)
  internal_mem(6) <= "100" & "0" & "000" & "1100" & "1100"; --E6: S1 S2 U1
     U2; incondicional carga E0
  internal_mem(7) <= "100" & "0" & "000" & "0011" & "0011"; --E7: no usado;
     regresa a E0 en reposo

  process(dir, internal_mem)
  begin
    data <= internal_mem(conv_integer(unsigned(dir)));
  end process;
end Behavioral;
```

Figura 5: Código VHDL de la memoria con las ocho localidades del direccionamiento implícito.

El módulo `divisor_datos` separa la palabra leída en sus campos.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity divisor_datos is
  Port( entrada : in STD_LOGIC_VECTOR(14 downto 0);
      prueba : out STD_LOGIC_VECTOR(2 downto 0);    --Selector de la variable
         a sensar
      VF : out STD_LOGIC;                           --Valor con el que se
         carga la liga
```

```
      liga : out STD_LOGIC_VECTOR(2 downto 0);      --Estado siguiente en
         caso de carga
      salidas_F : out STD_LOGIC_VECTOR(3 downto 0); --Salidas con la variable
         en 0
      salidas_V : out STD_LOGIC_VECTOR(3 downto 0)  --Salidas con la variable
         en 1
      );
end divisor_datos;

architecture Behavioral of divisor_datos is
begin
  process (entrada)
  begin
    prueba    <= entrada(14 downto 12);
    VF        <= entrada(11);
    liga      <= entrada(10 downto 8);
    salidas_F <= entrada(7 downto 4);
    salidas_V <= entrada(3 downto 0);
  end process;
end Behavioral;
```

Figura 6: Código VHDL del divisor que separa la palabra en prueba, VF, liga y salidas.

El módulo `mux5` es el selector de entradas. Su salida es la variable sensada, y la entrada `aux` permite los saltos incondicionales.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

--Selector de la variable a sensar, gobernado por la parte de prueba
entity mux5 is
  Port( sel : in STD_LOGIC_VECTOR(2 downto 0);
      --defina sus entradas
      Q0 : in STD_LOGIC;   --X
      Q1 : in STD_LOGIC;   --Y
      Q2 : in STD_LOGIC;   --Z
      Q3 : in STD_LOGIC;   --W
      aux : in STD_LOGIC;  --Constante (GND) para transiciones incondicionales
      O : out STD_LOGIC
      );
end mux5;

architecture Behavioral of mux5 is
begin
  process (sel, Q0, Q1, Q2, Q3, aux)
```

```
  begin
    case sel is
      when "000" => O <= Q0;
      when "001" => O <= Q1;
      when "010" => O <= Q2;
      when "011" => O <= Q3;
      when others => O <= aux;
    end case;
  end process;
end Behavioral;
```

Figura 7: Código VHDL del selector de entradas.

El módulo `mux1X2X1` selecciona entre las salidas F y V, y `mux1X2X1_3` selecciona entre el incremento y la liga del contador.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity mux1X2X1 is
  Port( sel : in STD_LOGIC;
      E0 : in STD_LOGIC_VECTOR(3 downto 0); -- Salidas Falsas
      E1 : in STD_LOGIC_VECTOR(3 downto 0); -- Salidas Verdaderas
      O : out STD_LOGIC_VECTOR(3 downto 0)
      );
end mux1X2X1;

architecture Behavioral of mux1X2X1 is
begin
  process (sel, E0, E1)
  begin
    if sel = '0' then
      O <= E0;
    elsif sel = '1' then
      O <= E1;
    end if;
  end process;
end Behavioral;
```

Figura 8: Código VHDL del selector de salidas (4 bits).

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity mux1X2X1_3 is
```

```
  Port( sel : in STD_LOGIC; -- Carga
      --Definir ligas de incremento y carga
      E0 : in STD_LOGIC_VECTOR(2 downto 0); -- Liga de incremento (edo + 1)
      E1 : in STD_LOGIC_VECTOR(2 downto 0); -- Liga de carga (viene de
         memoria)
      salida : out STD_LOGIC_VECTOR(2 downto 0) -- Estado Siguiente
      );
end mux1X2X1_3;

architecture Behavioral of mux1X2X1_3 is
begin
  process (sel, E0, E1)
  begin
    if sel = '0' then
      salida <= E0;
    else
      salida <= E1;
    end if;
  end process;
end Behavioral;
```

Figura 9: Código VHDL del selector de dirección (3 bits).

El módulo `incrementador` calcula el estado presente más uno.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity incrementador is
  Port( entrada : in STD_LOGIC_VECTOR(2 downto 0);
      salida : out STD_LOGIC_VECTOR(2 downto 0)
      );
end incrementador;

architecture Behavioral of incrementador is
begin
  process (entrada)
  begin
    --asignar a la salida el estado presente mas uno
    salida <= entrada + 1;
  end process;
end Behavioral;
```

Figura 10: Código VHDL del incrementador.

El módulo `register3` almacena el estado presente, con reset asíncrono activo en bajo.

```vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity register3 is
  Port( CLK : in STD_LOGIC;
      RESET : in STD_LOGIC;
      DATA_IN : in STD_LOGIC_VECTOR(2 downto 0); --Liga
      DATA_OUT : out STD_LOGIC_VECTOR(2 downto 0)--Edo Presente
      );
end register3;

architecture Behavioral of register3 is
  signal internal_value : std_logic_vector(2 downto 0) := B"000";
begin
  process (CLK, RESET, DATA_IN)
  begin
    if RESET = '0' then
      internal_value <= B"000";
    elsif rising_edge (CLK) then
      internal_value <= DATA_IN;
    end if;
  end process;

  process(internal_value)
  begin
    DATA_OUT <= internal_value;
  end process;
end Behavioral;
```

Figura 11: Código VHDL del registro de estado.

El módulo `contador` es el elemento secuencial del lazo. En cada flanco de reloj carga la liga si la señal de carga está activa, o incrementa en uno en caso contrario.

```vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

-- Contador de estado del direccionamiento implicito.
-- Reproduce el diagrama contador.bdf: el registro guarda el estado presente,
-- el incrementador calcula edo+1 y el multiplexor de 3 bits elige entre ese
-- incremento (Carga = '0') y la liga que entrega la memoria (Carga = '1').

```

```vhdl
entity contador is
  Port( Carga : in STD_LOGIC;
      Liga : in STD_LOGIC_VECTOR(2 downto 0);
      CLK : in STD_LOGIC;
      RESET : in STD_LOGIC;
      SAL : out STD_LOGIC_VECTOR(2 downto 0)
      );
end contador;

architecture Estructural of contador is

  component incrementador is
    Port( entrada : in STD_LOGIC_VECTOR(2 downto 0);
        salida : out STD_LOGIC_VECTOR(2 downto 0));
  end component;

  component mux1X2X1_3 is
    Port( sel : in STD_LOGIC;
        E0 : in STD_LOGIC_VECTOR(2 downto 0);
        E1 : in STD_LOGIC_VECTOR(2 downto 0);
        salida : out STD_LOGIC_VECTOR(2 downto 0));
  end component;

  component register3 is
    Port( CLK : in STD_LOGIC;
        RESET : in STD_LOGIC;
        DATA_IN : in STD_LOGIC_VECTOR(2 downto 0);
        DATA_OUT : out STD_LOGIC_VECTOR(2 downto 0));
  end component;

  signal edo_pres : std_logic_vector(2 downto 0);
  signal incremento : std_logic_vector(2 downto 0);
  signal edo_sig : std_logic_vector(2 downto 0);

begin

  U1 : incrementador
    port map( entrada => edo_pres,
           salida => incremento);

  U2 : mux1X2X1_3
    port map( sel => Carga,
           E0 => incremento,
           E1 => Liga,
```

```
            salida => edo_sig);

  U3 : register3
    port map( CLK => CLK,
            RESET => RESET,
            DATA_IN => edo_sig,
            DATA_OUT => edo_pres);

  SAL <= edo_pres;

end Estructural;
```

Figura 12: Código VHDL del contador con carga e incremento.

Para observar el comportamiento en la tarjeta, un divisor de frecuencia reduce el reloj de 50 MHz a una frecuencia de transición perceptible.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity div_frec is
  Port (reloj : in std_logic;
      div_clk : out std_logic);
end div_frec;

architecture Behavioral of div_frec is
begin
  process (reloj)
  variable cuenta : std_logic_vector (27 downto 0) := X"0000000";
  begin
    if rising_edge (reloj) then

      --if cuenta 0 "4000000" then
      if cuenta = X"2FAF080" then
        cuenta := X"0000000";
      else
        cuenta := cuenta+1;
      end if;
    end if;
    div_clk <= cuenta(25);
  end process;
end Behavioral;
```

Figura 13: Código VHDL del divisor de frecuencia.

Finalmente, la entidad `practica5` instancia los módulos anteriores y los interconecta, además de exponer el estado presente hacia los LEDs para poder observarlo.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

-- Practica 5: Construccion de Maquinas de Estados usando Memorias
-- Direccionamiento Implicito
--
-- Reproduce el diagrama de bloques de la profesora. La direccion de memoria es
-- unicamente el estado presente (3 bits). La palabra trae la parte de prueba, el
-- bit VF, una sola liga y los dos juegos de salidas. El selector mux5 engancha la
-- variable a sensar; Carga = XNOR(variable, VF) decide si el contador incrementa
-- (Carga='0') o carga la liga (Carga='1'). El mux de salidas conmuta con el valor
-- crudo de la variable sensada, no con Carga.

entity practica5 is
  Port( CLK : in STD_LOGIC;                          --Reloj de 50 MHz de la tarjeta
        RESET : in STD_LOGIC;                        --Activo en bajo (KEY0)
        X : in STD_LOGIC;
        Y : in STD_LOGIC;
        Z : in STD_LOGIC;
        W : in STD_LOGIC;
        S1 : out STD_LOGIC;
        S2 : out STD_LOGIC;
        U1 : out STD_LOGIC;
        U2 : out STD_LOGIC;
        EDO : out STD_LOGIC_VECTOR(2 downto 0)     --Estado presente
        );
end practica5;

architecture Estructural of practica5 is

  component div_frec is
    Port( reloj : in std_logic;
          div_clk : out std_logic);
  end component;
```

```
35
36  component contador is
37    Port( Carga : in STD_LOGIC;
38        Liga : in STD_LOGIC_VECTOR(2 downto 0);
39        CLK : in STD_LOGIC;
40        RESET : in STD_LOGIC;
41        SAL : out STD_LOGIC_VECTOR(2 downto 0));
42  end component;
43
44  component memory is
45    Port( dir : in STD_LOGIC_VECTOR(2 downto 0);
46        data : out STD_LOGIC_VECTOR(14 downto 0));
47  end component;
48
49  component divisor_datos is
50    Port( entrada : in STD_LOGIC_VECTOR(14 downto 0);
51        prueba : out STD_LOGIC_VECTOR(2 downto 0);
52        VF : out STD_LOGIC;
53        liga : out STD_LOGIC_VECTOR(2 downto 0);
54        salidas_F : out STD_LOGIC_VECTOR(3 downto 0);
55        salidas_V : out STD_LOGIC_VECTOR(3 downto 0));
56  end component;
57
58  component mux5 is
59    Port( sel : in STD_LOGIC_VECTOR(2 downto 0);
60        Q0 : in STD_LOGIC;
61        Q1 : in STD_LOGIC;
62        Q2 : in STD_LOGIC;
63        Q3 : in STD_LOGIC;
64        aux : in STD_LOGIC;
65        O : out STD_LOGIC);
66  end component;
67
68  component mux1X2X1 is
69    Port( sel : in STD_LOGIC;
70        E0 : in STD_LOGIC_VECTOR(3 downto 0);
71        E1 : in STD_LOGIC_VECTOR(3 downto 0);
72        O : out STD_LOGIC_VECTOR(3 downto 0));
73  end component;
74
75  signal clk_fsm : std_logic;
76  signal edo_pres : std_logic_vector(2 downto 0);
77  signal palabra_mem : std_logic_vector(14 downto 0);  --Contenido de memoria
      leido
```

```
78    signal prueba : std_logic_vector(2 downto 0);
79    signal VF : std_logic;
80    signal liga : std_logic_vector(2 downto 0);
81    signal salidas_F : std_logic_vector(3 downto 0);
82    signal salidas_V : std_logic_vector(3 downto 0);
83    signal var_sensada : std_logic;                    --Salida del mux5
84    signal carga : std_logic;                          --XNOR(var_sensada, VF)
85    signal salidas : std_logic_vector(3 downto 0);     --S1 S2 U1 U2
86
87  begin
88
89    U1_div : div_frec
90      port map( reloj => CLK,
91              div_clk => clk_fsm);
92
93    U2_cont : contador
94      port map( Carga => carga,
95              Liga => liga,
96              CLK => clk_fsm,
97              RESET => RESET,
98              SAL => edo_pres);
99
100   U3_mem : memory
101     port map( dir => edo_pres,
102             data => palabra_mem);
103
104   U4_div : divisor_datos
105     port map( entrada => palabra_mem,
106             prueba => prueba,
107             VF => VF,
108             liga => liga,
109             salidas_F => salidas_F,
110             salidas_V => salidas_V);
111
112   U5_mux : mux5
113     port map( sel => prueba,
114             Q0 => X,
115             Q1 => Y,
116             Q2 => Z,
117             Q3 => W,
118             aux => '0',
119             O => var_sensada);
120
121   --Compuerta XNOR del diagrama: carga la liga cuando la variable sensada
```

```
122   --coincide con VF; en cualquier otro caso el contador incrementa.
123   carga <= var_sensada xnor VF;
124
125   U6_sal : mux1X2X1
126     port map( sel => var_sensada,
127             E0 => salidas_F,
128             E1 => salidas_V,
129             O => salidas);
130
131   S1 <= salidas(3);
132   S2 <= salidas(2);
133   U1 <= salidas(1);
134   U2 <= salidas(0);
135   EDO <= edo_pres;
136
137 end Estructural;
```

Figura 14: Código VHDL de la entidad de nivel superior que interconecta los módulos.

Asignación de pines

Para la implementación en la tarjeta DE10-Lite (dispositivo MAX 10 10M50DAF484C7G) se realizó la asignación de la tabla 4. Las entradas X, Y, Z y W se asignaron a los interruptores SW0 a SW3, el *reset* al botón `KEY0`, las salidas S_1, S_2, $\overline{U_1}$ y $\overline{U_2}$ a `LEDR0` a `LEDR3`, y los tres bits del estado presente a `LEDR9`, `LEDR8` y `LEDR7`.

Tabla 4: Asignación de pines en la tarjeta DE10-Lite.

Señal	Pin	Recurso	Función
CLK	PIN_P11	MAX10_CLK1_50	Reloj de 50 MHz
RESET	PIN_B8	KEY0	*Reset* activo en bajo
X	PIN_C10	SW0	Entrada X
Y	PIN_C11	SW1	Entrada Y
Z	PIN_D12	SW2	Entrada Z
W	PIN_C12	SW3	Entrada W (no usada por la carta)
S1	PIN_A8	LEDR0	Salida S_1 (activa en 1)
S2	PIN_A9	LEDR1	Salida S_2 (activa en 1)
U1	PIN_A10	LEDR2	Salida $\overline{U_1}$ (activa en 0)
U2	PIN_B10	LEDR3	Salida $\overline{U_2}$ (activa en 0)
EDO[2]	PIN_B11	LEDR9	Bit 2 del estado presente
EDO[1]	PIN_A11	LEDR8	Bit 1 del estado presente
EDO[0]	PIN_D14	LEDR7	Bit 0 del estado presente

Análisis de resultados

El proyecto se compiló en Quartus Prime 18.1.0 Lite Edition sin errores. La tabla 5 resume los recursos utilizados y el desempeño reportado por la herramienta.

Tabla 5: Resumen de compilación del proyecto en Quartus Prime 18.1.

Parámetro	**Valor**
Elementos lógicos	62 / 49 760 (< 1 %)
Registros dedicados	31 / 49 760 (< 1 %)
Pines de E/S	13 / 360 (4 %)
Bits de memoria embebida	0 / 1 677 312 (0 %)
F_{max} (modelo 1200 mV, 85 °C)	168.75 MHz
Errores de compilación	0

Dos cifras de la tabla 5 merecen comentario. La primera es el conteo de 31 registros: 28 corresponden al contador del divisor de frecuencia y los 3 restantes al registro de estado, lo que confirma que el único elemento secuencial del lazo de la máquina es ese registro. La segunda es que la herramienta reporta cero bits de memoria embebida, ya que la ROM de 120 bits no se mapeó a los bloques M9K del dispositivo sino que se sintetizó como lógica combinacional dentro de los 62 elementos lógicos; esto es razonable dado su tamaño y no altera el comportamiento del circuito. Los 168.75 MHz de frecuencia máxima superan ampliamente los 50 MHz del oscilador, de modo que el diseño opera con holgura; la advertencia de incumplimiento temporal que emite el analizador se debe únicamente a que el proyecto no incluye un archivo `.sdc`, y en ausencia de restricciones Quartus asume un requisito por omisión de 1 GHz.

Antes de programar la tarjeta se verificó el comportamiento del diseño mediante simulación funcional.

Simulación funcional

La simulación se compiló y ejecutó con GHDL, y la forma de onda se visualizó con GTKWave a partir del archivo `sim.vcd`. El *testbench* `tb_practica5` instancia la cadena funcional (memoria, divisor de datos, selector de entradas, selectores de salida y de dirección, y el contador) y maneja las entradas X, Y, Z y W. Al ejecutarse, el banco de pruebas aplica 18 transiciones y compara cada una con la carta ASM mediante sentencias `assert`. Al finalizar reporta: *OK: las 18 transiciones y sus salidas coinciden con la carta ASM*.

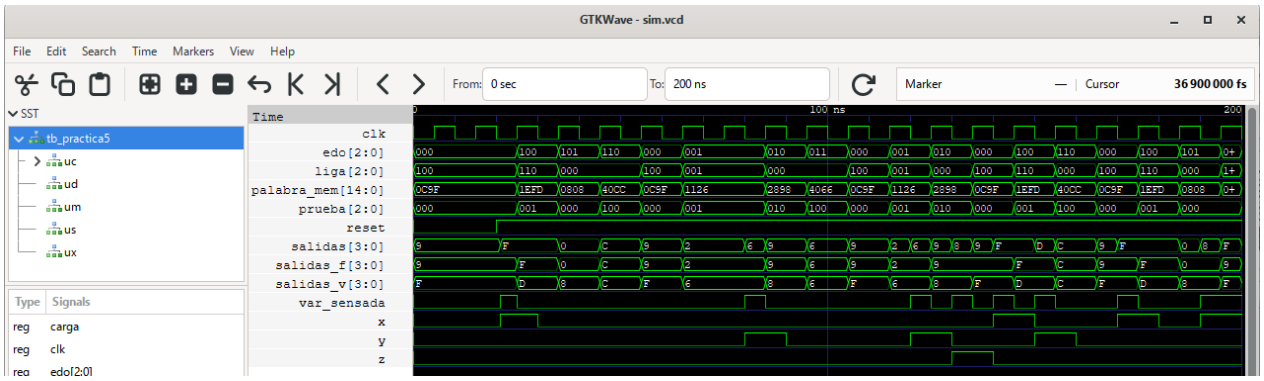


Figura 15: Forma de onda de la simulación funcional. Se muestran el reloj, las entradas, el estado presente (`edo`), la palabra leída de memoria (`palabra_mem`), la parte de prueba, la liga, la variable sensada y los tres juegos de salidas durante las 18 transiciones.

Verificación en la tarjeta

Se programó la DE10-Lite con el diseño y se comprobó su funcionamiento en el laboratorio. La máquina recorrió los estados de la carta ASM conforme a las condiciones de cada decisión, y las salidas condicionales se activaron en los estados correspondientes. El funcionamiento se mostró a la profesora durante la sesión.

Conclusiones individuales

Omar Alfonso Zendejas Labias

Con esta práctica entendí que el direccionamiento implícito cambia la forma en que la memoria decide el siguiente estado. En lugar de guardar dos ligas, como en el método anterior, la palabra guarda una sola liga y deja que el contador avance en uno cuando la condición no se cumple. Lo que más me sirvió fue ver que la comparación entre la variable sensada y VF, hecha con una XNOR, es la que decide entre incrementar y cargar, y que con ese mecanismo basta una sola liga por estado. También me quedó claro que la carta debe construirse con los códigos consecutivos en las ramas de incremento, porque de lo contrario la memoria no puede representar la transición.
Un punto que me ayudó a entender el método fue el error en mi cálculo manual. Al principio estimé 112 bits sin considerar los saltos incondicionales de EST3 y EST6. Al revisar la carta me di cuenta de que esas transiciones necesitaban una entrada fija, y por eso el campo de prueba pasó a tener 3 bits y la memoria quedó en 120 bits. Esa corrección me mostró que el tamaño de la memoria depende no solo de los estados y entradas, sino también de cómo se resuelven las transiciones sin condición.

Emiliano García Olvera

Considero que el objetivo de la práctica se cumplió. Con el direccionamiento implícito se implementó una máquina de siete estados con tres entradas y cuatro salidas, usando una memoria de 8 localidades de 15 bits, es decir, 120 bits en total (tabla 3). La simulación con 18 transiciones no presentó discrepancias respecto a la carta ASM, y el funcionamiento en la tarjeta coincidió con la simulación, lo que confirma que el contenido de la memoria y la lógica de carga e incremento son consistentes entre sí. Lo que más me aportó la práctica fue entender que las salidas condicionales se obtienen desdoblando el campo de salidas en dos conjuntos y seleccionando uno de ellos con el valor de la variable sensada, la misma que alimenta la XNOR que decide el siguiente estado.

Bibliografía

[1] J. Savage Carmona, G. Vázquez y N. E. Chávez Rodríguez, *Diseño de microprocesadores*. Ciudad de México, México: Facultad de Ingeniería, Universidad Nacional Autónoma de México, 2017. [En línea]. Disponible: `http://profesores.fi-b.unam.mx/luist/archivos/MicroprocesadoresSavage.pdf`

[2] M. M. Mano y M. D. Ciletti, *Diseño digital*, 5a ed. Naucalpan de Juárez, México: Pearson Educación, 2013.